# Remix, Don't Expand: Chunked Template Routing for Compute-Efficient Transformers

**Anonymous ACL submission**

## Abstract

Scaling Transformer language models by widening dense linear layers increases both parameter count and inference cost proportionally. Mixture-of-Experts (MoE) architectures decouple these by routing tokens to discrete expert subnetworks, but introduce load-balancing complexity and often underperform dense baselines at small-to-medium scale. We introduce **RemixedLinear**, a drop-in replacement for standard linear layers that dynamically mixes $K$ learned template matrices per chunk of tokens using lightweight content-aware routing. Each token's effective linear operator is a context-dependent combination of shared templates, gated by a centered output gate $(1 + \tanh)$ that preserves dense-parity gradients from initialization. Routing is amortized over 64-token chunks and balanced via quantile tracking, eliminating the auxiliary losses required by standard MoE. Through 29 phases of systematic ablation, we identify the key design choices that enable dynamic linear layers to outperform static baselines: chunk-amortized routing, quantile balancing, and identity-preserving initialization. At depth 12 ($\sim$290M active parameters), RemixedLinear achieves **0.814 bits-per-byte** compared to 0.903 for the dense baseline at matched active FLOPs, a 10% improvement. Standard MoE with equivalent routing underperforms the dense baseline. These results hold across three model scales (d4, d8, d12) following a consistent power-law improvement over dense scaling curves.

## 1 Introduction

Linear transformations dominate both the parameter count and computational cost of Transformer language models (Vaswani et al., 2017). As models scale, improving the efficiency of these layers becomes critical for practical deployment. Two broad strategies have emerged: static compression methods such as low-rank factorization (Hu et al., 2022), which reduce parameters but restrict all inputs to a fixed subspace; and sparse routing methods such as Mixture-of-Experts (Shazeer et al., 2017; Fedus et al., 2022), which activate only a subset of parameters per token but introduce routing instability, load-balancing overhead, and often require substantially larger total parameter counts.

We introduce **RemixedLinear**, a linear layer that stores $K$ learned template matrices and dynamically composes them per chunk of tokens using content-aware routing. Unlike standard MoE, which routes entire tokens to discrete expert subnetworks at the feed-forward network (FFN) level, RemixedLinear operates at the level of individual linear projections (Q, K, V, projection, and both FFN layers), providing finer-grained dynamic computation. Unlike static low-rank layers, each token induces a distinct full-rank effective operator via continuous template mixing.

The design of RemixedLinear was shaped by 29 phases of systematic ablation. Early designs based on content-dependent multiplicative gating consistently failed due to what we term *optimization friction*: the gradient cost of simultaneously learning routing and feature weights exceeded the capacity benefit of dynamic computation. The key innovations that resolved this are: (1) **chunk routing**, which amortizes a single routing decision over $N{=}64$ tokens, reducing per-token routing overhead and stabilizing gradients; (2) **quantile-balanced routing**, which replaces auxiliary load-balancing losses with EMA-tracked quantile normalization; and (3) a **centered output gate** $(1 + \tanh)$, initialized to pass-through, ensuring the model starts with dense-equivalent gradients.

We evaluate RemixedLinear on the nanochat framework (Karpathy, 2025) using the ClimbMix dataset with Chinchilla-optimal training horizons (Hoffmann et al., 2022). At depth 12 ($\sim$290M active parameters), RemixedLinear achieves 0.814 bits-per-byte (BPP) compared to 0.903 for the dense baseline at matched active FLOPs. Standard

MoE at depth 4 scores 1.187 BPP, worse than the dense baseline of 1.17. These results hold across three model scales, with RemixedLinear points consistently falling below the dense power-law scaling curve when plotted against active compute.

## 2 Related Work

**Mixture-of-Experts.** MoE architectures (Shazeer et al., 2017; Lepikhin et al., 2021; Fedus et al., 2022) route tokens to discrete expert subnetworks, enabling large parameter counts with constant inference FLOPs. However, MoE introduces routing instability, expert collapse, and hardware inefficiency from sparse memory access. Expert Choice routing (Zhou et al., 2022) and ST-MoE (Zoph et al., 2022) address load balancing but retain the coarse granularity of routing entire tokens to complete FFN blocks. RemixedLinear shares the goal of decoupling stored and active parameters but operates at finer granularity (individual linear projections) and uses continuous mixing rather than discrete routing.

**Dynamic Weights and Hypernetworks.** Hypernetworks (Ha et al., 2017) generate weights conditioned on context but require $O(d^2)$ operations per token to produce a full weight matrix. Fast Weight Programmers (Schlag et al., 2021) explore similar principles for attention. RemixedLinear generates only $O(K)$ routing coefficients to mix pre-existing template matrices, achieving dynamic weights with negligible overhead.

**Parameter-Efficient Adaptation.** LoRA (Hu et al., 2022) uses static low-rank decompositions for efficient fine-tuning. RemixedLinear extends this principle to the dynamic regime: the basis projection provides compression while template routing provides token-dependent expressivity that static decompositions cannot achieve.

**Gated Linear Units.** GLU variants (Dauphin et al., 2017; Shazeer, 2020) gate neuron activations after a fixed linear transformation. RemixedLinear gates at a different level: it modulates which *template combination* constructs the effective linear operator, then applies a separate low-rank output gate. This produces a family of context-dependent transformations rather than a single gated projection.

**Scaling Laws.** Chinchilla scaling laws (Hoffmann et al., 2022) establish compute-optimal training horizons as a function of model size. We adopt this framework, training each model with a fixed token-to-parameter ratio, enabling fair comparison across scales. Our results demonstrate that RemixedLinear shifts the scaling curve, achieving lower BPP at matched active FLOPs.

## 3 Method

### 3.1 Overview

A standard linear layer computes $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, where $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is a static matrix applied identically to all inputs. RemixedLinear replaces this with a dynamic composition:

$$\mathbf{y} = (\mathbf{W}_{\text{eff}}(\mathbf{c}) \cdot \text{LN}(\mathbf{W}_b \mathbf{x})) \odot \mathbf{g}_{\text{out}}(\mathbf{c}) + \mathbf{b} \quad (1)$$

where $\mathbf{W}_b \in \mathbb{R}^{B \times d_{\text{in}}}$ is a basis projection ($B$ = basis size), $\mathbf{W}_{\text{eff}}(\mathbf{c})$ is a context-dependent effective mixing matrix constructed from $K$ templates, and $\mathbf{g}_{\text{out}}(\mathbf{c})$ is a low-rank output gate. The context signal $\mathbf{c}$ is derived from a causal context stream (Section 3.4).

### 3.2 Template Mixing with Chunk Routing

RemixedLinear stores $K$ template matrices $\{\mathbf{T}_k\}_{k=1}^{K}$, each $\mathbf{T}_k \in \mathbb{R}^{d_{\text{out}} \times B}$. The effective mixing matrix is a weighted combination:

$$\mathbf{W}_{\text{eff}}(\mathbf{c}) = \sum_{k=1}^{K} \alpha_k(\mathbf{c}) \cdot \mathbf{T}_k \quad (2)$$

where $\boldsymbol{\alpha}(\mathbf{c}) = \text{softmax}(\mathbf{W}_r \mathbf{c}/\tau)$ are routing weights computed from the context signal, $\mathbf{W}_r \in \mathbb{R}^{K \times d_{\text{in}}}$ is a learned routing projection, and $\tau$ is a temperature parameter.

**Chunk amortization.** Per-token routing introduces two problems: high FLOPs overhead from computing $\boldsymbol{\alpha}$ for every token, and gradient instability from the multiplicative coupling between routing weights and template gradients. We amortize routing over chunks of $N$=64 tokens: the routing weights $\boldsymbol{\alpha}$ are computed once from the mean context of the chunk and shared across all tokens in that chunk. This reduces routing FLOPs by a factor of $N$ and stabilizes training by decoupling routing updates from per-token gradient noise.

**Quantile-balanced routing.** Standard MoE requires auxiliary load-balancing losses to prevent expert collapse (Fedus et al., 2022). We replace this with quantile-balanced routing: an EMA tracks the

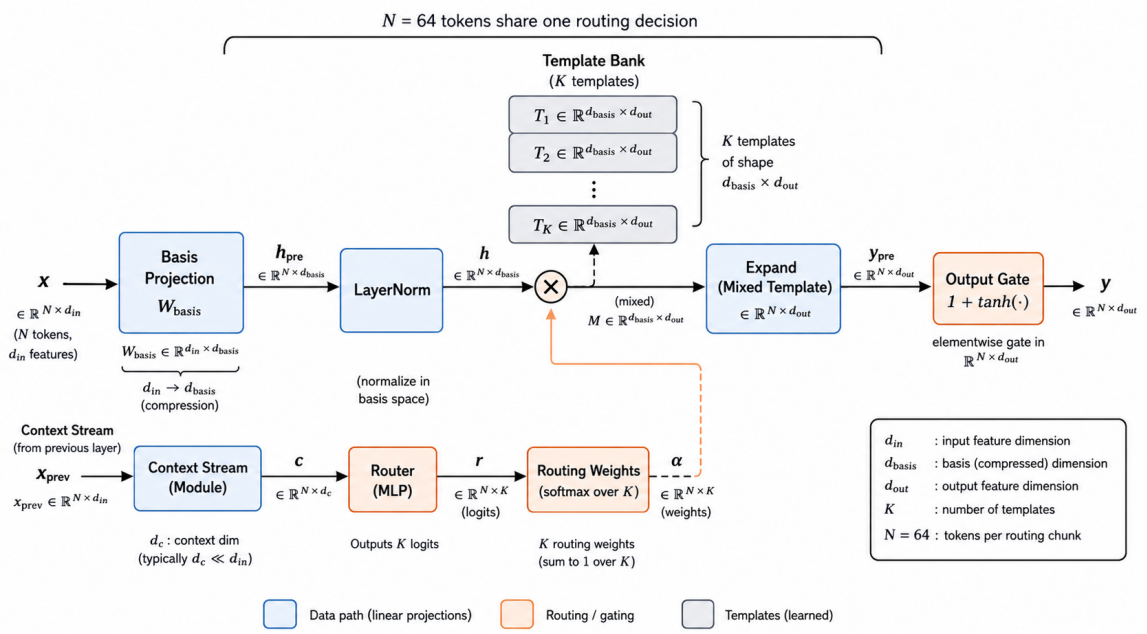


Figure 1: Architecture of the RemixedLinear layer. The data path (blue) compresses input $\mathbf{x}$ via a basis projection, normalizes it, then expands through a mixed template matrix. The context stream (orange) produces routing weights $\boldsymbol{\alpha}$ that select the template combination from a bank of $K$ learned templates (gray). Routing is amortized over chunks of $N$=64 tokens. A centered output gate $(1 + \tanh)$ modulates the final output.

running distribution of routing logits, and logits are normalized to quantile space before the softmax. This ensures balanced template utilization without introducing an additional loss term or its associated hyperparameter.

### 3.3 Centered Output Gate

A low-rank output gate modulates the output:

$$\mathbf{g}_{\text{out}}(\mathbf{c}) = 1 + \tanh(s \cdot (\mathbf{W}_c \mathbf{c})\mathbf{G}) \tag{3}$$

where $\mathbf{W}_c \in \mathbb{R}^{r \times d_{\text{in}}}$, $\mathbf{G} \in \mathbb{R}^{r \times d_{\text{out}}}$, $r$=8, and $s$ is a learned scalar initialized to 0.1. The $1 + \tanh(\cdot)$ formulation, with $\mathbf{W}_c$ and $\mathbf{G}$ zero-initialized, ensures $\mathbf{g}_{\text{out}} = 1$ at initialization. This is critical: it guarantees that RemixedLinear produces identical gradients to a standard linear layer at step 0, avoiding the optimization friction that caused all sigmoid-gated designs to fail in our ablation studies (Appendix A).

### 3.4 Context Stream

The context signal $\mathbf{c}$ is derived from a causal context stream that maintains a running representation across the sequence. We use a *selective* stream with GRU-style gating:

$$\mathbf{c}_t = (1 - \mathbf{z}_t) \odot \mathbf{c}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{c}}_t \tag{4}$$

where $\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t)$ is an input-dependent update gate and $\tilde{\mathbf{c}}_t$ is a candidate state. The context state is *detached* between steps to prevent gradient explosion through long recurrent chains, a lesson from Phase 3 of our ablation study (Appendix A).

### 3.5 Integration into the Transformer Block

RemixedLinear replaces all six linear projections in each Transformer block: Q, K, V, attention output projection, and both FFN layers ($\mathbf{W}_{\text{fc}}$, $\mathbf{W}_{\text{proj}}$). Each projection has its own set of $K$ templates and routing weights, allowing different projections to specialize independently. The context stream is shared within each block.

### 3.6 Complexity Analysis

Table 1 compares per-token FLOPs for a single projection.

With $B = d$ (full-rank basis), RemixedLinear uses approximately $4d^2$ FLOPs per projection versus $2d^2$ for dense. However, the model stores $K$ template matrices totaling $K \cdot B \cdot d$ parameters while the active computation per token uses only the weighted sum. The ratio of stored to active capacity is approximately $K$, enabling the model to express a richer family of transformations from a larger parameter bank while maintaining bounded inference cost.

Table 1: FLOPs per token for a single linear projection with $d_{\text{in}} = d_{\text{out}} = d$, basis size $B = d$, $K$ templates, chunk size $N$.

| Component | FLOPs |
|---|---|
| Dense baseline ($\mathbf{W}\mathbf{x}$) | $2d^2$ |
| Basis projection ($\mathbf{W}_b\mathbf{x}$) | $2dB$ |
| Template mixing ($\mathbf{W}_{\text{eff}}\mathbf{h}$) | $2Bd$ |
| Routing (amortized, $1/N$) | $2dK/N$ |
| Output gate (rank $r$) | $2(dr + rd)$ |
| **Total (RemixedLinear)** | $\approx 4dB + 32d$ |

Table 2: Model configurations. RemixedLinear uses $K$=8 templates, chunk size $N$=64, quantile routing, and selective context stream.

| | d4 | d8 | d12 |
|---|---|---|---|
| Model dim | 256 | 512 | 768 |
| Heads | 2 | 4 | 6 |
| Dense params | ∼37M | ∼126M | ∼286M |
| Remix total params | 56M | 276M | 792M |
| Remix active params | 37M | 127M | 290M |
| Training tokens | ∼0.4B | ∼1.3B | ∼3.0B |

# 4 Experiments

## 4.1 Setup

**Framework.** All experiments use nanochat (Karpathy, 2025), a single-file GPT implementation supporting Flash Attention (Dao et al., 2022), bf16 mixed precision, and the Muon optimizer (Jordan et al., 2025) for matrix-shaped parameters with AdamW for embeddings and scalars.

**Dataset.** ClimbMix (NVIDIA, 2024), a curated pretraining mixture, tokenized with a 32,768-vocabulary BPE tokenizer. Sequence length is 2048 tokens.

**Training horizon.** Following Chinchilla scaling (Hoffmann et al., 2022), each model is trained with a fixed 10.5× token-to-scaling-parameter ratio, where scaling parameters exclude embedding tables. This ensures compute-optimal training at every scale.

**Metric.** Validation bits-per-byte (BPP), a vocabulary-size-invariant measure of language model quality. Lower is better.

**Model scales.** We evaluate at three depths with model dimension $d = \text{depth} \times 64$, rounded to the nearest multiple of head dimension (128):

**Baselines.** (1) Dense Transformer baseline (nanochat default) at depths 12 through 30. (2) Standard MoE with 8 experts, top-1 and top-all routing at depth 4.

Table 3: Main results. RemixedLinear consistently outperforms the dense baseline at matched active FLOPs across all three scales. Standard MoE underperforms the dense baseline at d4.

| Model | Total Params | Active Params | FLOPs | BPP |
|---|---|---|---|---|
| *Dense baselines* | | | | |
| Dense d4 | 37M | 37M | 7.8e7 | 1.170 |
| Dense d8 | 126M | 126M | 2.9e8 | 0.969 |
| Dense d12 | 286M | 286M | 7.6e8 | 0.903 |
| *RemixedLinear (ours)* | | | | |
| Remix d4 | 56M | 37M | 8.0e7 | **1.082** |
| Remix d8 | 276M | 127M | 2.9e8 | **0.903** |
| Remix d12 | 792M | 290M | 7.6e8 | **0.814** |
| *Standard MoE (d4 only)* | | | | |
| MoE top-1 | 51M | 37M | 7.8e7 | 1.187 |
| MoE top-all | 51M | 51M | 1.7e8 | 1.186 |

## 4.2 Main Results

Table 3 presents the primary comparison.

At every scale, RemixedLinear achieves lower BPP than the dense baseline at comparable active FLOPs. The improvement grows with scale: 7.5% at d4, 6.8% at d8, and 9.9% at d12. Notably, RemixedLinear at d8 (0.903 BPP, 127M active params) matches the dense d12 baseline (0.903 BPP, 286M params), achieving equivalent quality with 2.25× fewer active parameters.

Standard MoE at d4 performs *worse* than the dense baseline (1.187 vs. 1.170 BPP), despite having 51M total parameters. This suggests that coarse-grained expert routing at the FFN level is insufficient at this scale, while RemixedLinear's fine-grained template routing within individual projections provides a qualitatively different advantage.

## 4.3 Scaling Law Analysis

Figure 2 plots BPP against total parameters, active parameters, and active FLOPs. The dense baseline (d12–d30) follows a tight power law $\text{BPP} = a \cdot \text{FLOPs}^b$. RemixedLinear points fall consistently below this curve when measured against active FLOPs, confirming that the architecture achieves genuine compute efficiency gains rather than merely trading parameters for quality.

The d12 RemixedLinear point is particularly striking: at nearly identical active FLOPs to the dense d12 baseline ($7.58 \times 10^8$ vs. $7.60 \times 10^8$), it achieves 0.814 BPP versus 0.903, matching the

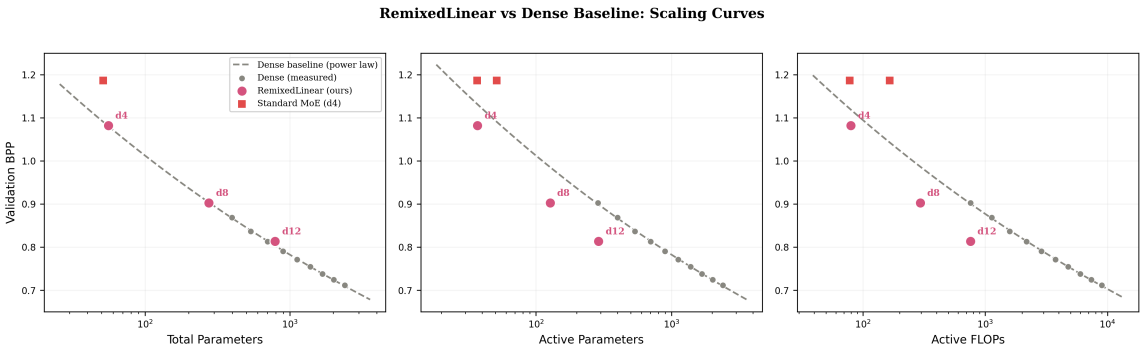


Figure 2: Validation BPP vs. total parameters (left), active parameters (center), and active FLOPs (right). Dashed lines show power-law fits to dense baselines (d12–d30). RemixedLinear points (pink circles) fall consistently below the dense scaling curve, indicating better compute efficiency. Standard MoE (red squares) falls above the curve at d4.

Table 4: Ablation study at d4. *Top*: routing variants with $K{=}8$ templates. *Middle*: gate component ablation with $K{=}1$ template (isolating gate effects from routing). *Bottom*: standard MoE baselines. $\Delta$ is relative to the dense baseline BPP of 1.170.

| **Configuration** | **BPP** | $\Delta$ |
|---|---|---|
| Dense baseline | 1.170 | – |
| *Routing variants ($K{=}8$ templates)* | | |
| 29A: Top-1 routing (per-token) | 1.087 | −7.1% |
| 29C: Chunk-64 routing | **1.082** | −7.5% |
| 29E: Top-1 + Chunk-64 | 1.082 | −7.5% |
| 29H: 4 templates, dense mix | 1.127 | −3.7% |
| *Gate component ablation ($K{=}1$)* | | |
| No context, no gates | 1.168 | −0.2% |
| Output gate + context only | 1.165 | −0.4% |
| + Linear basis gate | 1.160 | −0.9% |
| *Standard MoE ($K{=}8$ experts)* | | |
| MoE top-1 | 1.187 | +1.5% |
| MoE top-all | 1.186 | +1.4% |

dense power-law extrapolation at approximately d20 scale (∼900M params, ∼2.9 $\times 10^9$ FLOPs).

### 4.4 Ablation Study

To isolate the contribution of each design component, we run ablations at d4 using the P29 sweep configuration. Table 4 reports results for the key variants tested.

**Chunk routing matches per-token.** Configurations 29A (per-token top-1) and 29C (chunk-64) achieve nearly identical BPP (1.087 vs. 1.082), while chunk routing uses substantially fewer routing FLOPs. The combined variant (29E) shows no additional benefit, confirming that chunk-64 captures the useful routing structure.

**Template count matters.** Reducing from $K{=}8$ to $K{=}4$ templates (29H) degrades BPP from 1.082 to 1.127, though it still outperforms the dense baseline. This suggests the model benefits from a diverse set of template specializations.

**Gate contributions are modest but additive.** With a single template ($K{=}1$), the basis-mixing factorization alone already matches the dense baseline (1.168 vs. 1.170). Adding the output gate and context stream improves BPP to 1.165, and further adding a linear basis gate reaches 1.160. The basis gate thus contributes only 0.005 BPP improvement at significant FLOPs cost ($\sim 0.625d^2$ per projection); we omit it in the final architecture, relying on the output gate and context stream for modulation while template routing provides the primary performance gain.

**Standard MoE fails at this scale.** Both MoE variants underperform the dense baseline, indicating that coarse expert routing is insufficient when the individual expert capacity is limited.

## 5 Discussion

**Why fine-grained routing outperforms coarse MoE.** Standard MoE routes entire tokens to one of $K$ expert FFN blocks. At small-to-medium scale, each expert has limited capacity, and the discrete routing decision means most of the expert bank is unused per token. RemixedLinear's template mixing operates within each linear projection (six per block), providing $6 \times K$ effective routing decisions per block. The continuous softmax mixing ensures all templates contribute, avoiding the expert collapse problem. This fine-grained compo-

sition appears to be qualitatively superior to coarse-grained expert selection at the scales we tested.

**The optimization friction problem.** Our 29-phase ablation history (Appendix A) documents a systematic pattern in early designs (Phases 4–9): multiplicative sigmoid gating that modulated individual basis features through learned, per-token routing consistently degraded training relative to the dense baseline. The final template-mixing design resolves this: routing operates over $K$ template-level softmax weights rather than per-feature sigmoid gates, producing a much looser gradient coupling. Per-token learned routing (29A) works well in this formulation (1.087 BPP vs. 1.170 dense), though chunk-amortized routing (29C) achieves equivalent quality (1.082) at lower routing cost. The centered output gate ($1 + \tanh$, zero-initialized) further reduces friction by ensuring the model starts with dense-equivalent gradients.

**Limitations.** Our RemixedLinear scaling curve spans only three points (d4, d8, d12). While the trend is consistent, confirming these gains at d20+ scale ($\sim$1B active parameters) remains essential future work. The standard MoE comparison is limited to d4 due to compute constraints; extending this comparison to d8 and d12 would strengthen the conclusions. All experiments were conducted by an independent researcher without institutional compute resources, limiting the breadth of the scaling analysis. The total parameter count of RemixedLinear models is $K\times$ larger than dense equivalents; while active parameters and FLOPs are matched, memory footprint during training is higher.

## 6 Conclusion

We introduced RemixedLinear, a drop-in replacement for dense linear layers that dynamically composes $K$ template matrices per chunk of tokens using content-aware routing. Through 29 phases of ablation, we identified chunk-amortized routing, quantile balancing, and centered output gating as the critical design choices that enable dynamic linear layers to outperform static baselines. At d12 scale, RemixedLinear achieves 10% lower BPP than the dense baseline at matched active FLOPs, while standard MoE underperforms the baseline entirely. These results suggest that fine-grained template routing within linear projections provides a promising alternative to coarse expert routing for compute-efficient Transformers.


## Acknowledgements

In accordance with the AI Writing Assistance Policy, we acknowledge the use of large language models to assist with text editing. All content was reviewed and verified by the human author.


## References


Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35.

Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. 2017. Language modeling with gated convolutional networks. *arXiv preprint arXiv:1612.08083*.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

David Ha, Andrew Dai, and Quoc V Le. 2017. Hypernetworks. In *International Conference on Learning Representations*.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, and 1 others. 2022. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Keller Jordan, Yuqing Chen, Micah Goldblum, Nikunj Saunshi, and Preetum Nakkiran. 2025. Muon: An optimizer for hidden layers in neural networks. *arXiv preprint arXiv:2502.16982*.

Andrej Karpathy. 2025. nanochat: The best chatgpt that $100 can buy.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. Gshard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*.

NVIDIA. 2024. Climbmix: A curated pretraining dataset. Used via nanochat framework.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. 2021. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pages 9342–9351. PMLR.

Noam Shazeer. 2020. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35.

Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*.

## A Design Evolution: From CCL to RemixedLinear

The final RemixedLinear architecture emerged from 29 phases of systematic ablation, each testing a specific hypothesis about how to make context-conditioned linear layers work. This appendix summarizes the key phases and the lessons they produced.

**Phase 1–2: Global Context Manager.** The original design used a cross-attention mechanism to compute a global context vector for modulating all linear layers. This architecture showed strong early results but was later discovered to contain a **causal leakage bug**: the cross-attention looked into future tokens. Once the leak was fixed, the architecture failed to beat the dense baseline.

**Phase 3: BPTT through context.** Allowing gradient flow through the recurrent context stream (removing `.detach()`) caused gradient explosion. This established that **context states must be detached** between time steps.

**Phase 4–8: Activation vs. weight modulation.** DiT-style activation modulation (AdaRMSNorm), multi-scale temporal bottlenecks, cross-layer stale context, and boundary-gated updates were all tested. None matched the dense baseline. The core finding: **weight-level modulation is more expressive than activation-level modulation**, but the gating mechanism must not interfere with the primary gradient path.

**Phase 9: The identity trap.** When context is derived from the same $\mathrm{norm}(\mathbf{x})$ that the FFN processes, multiplicative sigmoid gating collapses to identity. Additive residual designs (RAL) also failed despite identity-safe initialization, establishing that **zero-init does not guarantee zero friction** during training.

**Phase 12–17: CKR and position routing.** Causal Kernel Reparameterization (CKR) using position-only routing was the first design to beat the dense baseline (+0.02 BPP). However, this improvement could not be reproduced reliably and did not scale beyond $K{=}4$ branches. The lesson: **position conditioning has a low ceiling**, as RoPE already provides adequate positional information.

**Phase 20–21: Frozen content routing.** Low-Rank Composition with Frozen Basis (LRCFB) used a frozen random projection for content-dependent routing, achieving $-0.025$ BPP improvement. This validated that **content routing works if the routing projection does not receive gradients from the main loss**.

**Phase 22–29: Template mixing and chunk routing.** Building on LRCFB, we introduced learned templates with chunk routing, quantile balancing, and the centered output gate. The $1 + \tanh$ gate formulation was the final key ingredient, resolving the optimization friction that had blocked all prior sigmoid-gated designs. The canonical configuration ($K{=}8$, chunk-64, quantile routing) is the subject of this paper.

**Summary of failure modes.**

- **FM1 (Gradient fracturing):** In early sigmoid-gated designs, learned per-feature routing coupled with feature weight gradients degraded training. The template softmax formulation avoids this by routing at the template level rather than the feature level.
- **FM2 (Identity trap):** Context derived from $\mathrm{norm}(\mathbf{x})$ collapses gating to identity.
- **FM3 (Rank bottleneck):** Subspace restrictions (low-rank deltas, channel splitting) underperform full-rank alternatives.
- **FM4 (Capacity tax):** At small scale, any parameter overhead for routing mechanics is proportionally too expensive.
- **FM5 (Zero-init illusion):** Starting as identity does not prevent the added pathway from disrupting training once gradients flow.

The resolution to all five failure modes was the combination of (1) template-level softmax routing (replacing per-feature sigmoid gating) to resolve FM1, with optional chunk amortization for efficiency, (2) a detached causal context stream to avoid FM2, (3) full-rank templates to avoid FM3, (4) the centered output gate to avoid FM4 and FM5.